

Documentation Technique — CesiZen

Projet : CesiZen — Application de bien-être et de gestion du stress

Version : 1.0.0

Date : Mai 2026

Auteur : Équipe de développement CesiZen

Table des matières

1. [Présentation du projet](#)
 2. [Architecture générale du système](#)
 3. [Comparatif des solutions techniques](#)
 4. [Pertinence de la solution retenue](#)
 5. [Modèle Logique de Données \(MLD\)](#)
 6. [Architecture backend détaillée](#)
 7. [Documentation de l'API REST](#)
 8. [Architecture mobile Flutter](#)
 9. [Sécurité](#)
 10. [Stratégie de tests](#)
 11. [Guide d'installation](#)
 12. [Annexes](#)
-

1. Présentation du projet

1.1 Contexte

CesiZen est une application mobile de bien-être mental et de gestion du stress développée dans le cadre d'un projet académique à CESI. Elle s'inscrit dans une démarche de sensibilisation à la santé mentale et propose des outils concrets permettant aux utilisateurs de mesurer leur niveau de stress, de suivre leurs émotions au quotidien et de découvrir des activités de mieux-être adaptées.

1.2 Objectifs fonctionnels

L'application répond à trois axes fonctionnels principaux :

Axe 1 — Diagnostic de stress

L'application intègre un questionnaire basé sur l'échelle de Holmes-Rahe, un outil cliniquement validé composé de 43 événements de vie auxquels sont associés des scores. À l'issue du questionnaire, l'utilisateur obtient une évaluation de son niveau de stress (faible, modéré ou élevé).

Axe 2 — Suivi émotionnel

Un tracker d'émotions permet à l'utilisateur d'enregistrer quotidiennement son état émotionnel parmi un référentiel d'émotions primaires (joie, peur, tristesse, colère, fatigue). Chaque entrée peut être accompagnée d'un commentaire libre.

Axe 3 — Activités bien-être

Un catalogue d'activités (respiration, méditation, yoga, etc.) est mis à disposition des utilisateurs. Ces activités sont enrichies de descriptions, de liens vidéo (YouTube) et de visuels. Les utilisateurs peuvent les mettre en favoris.

1.3 Périmètre technique

Le projet comporte deux interfaces distinctes :

- **Application utilisateur** : interface mobile Flutter destinée au grand public
- **Back-office administrateur** : interface Flutter séparée permettant la gestion du contenu et des utilisateurs

Un **serveur REST** Node.js/Express constitue la couche métier et expose l'ensemble des fonctionnalités via une API sécurisée. Une **base de données PostgreSQL** assure la persistance des données.

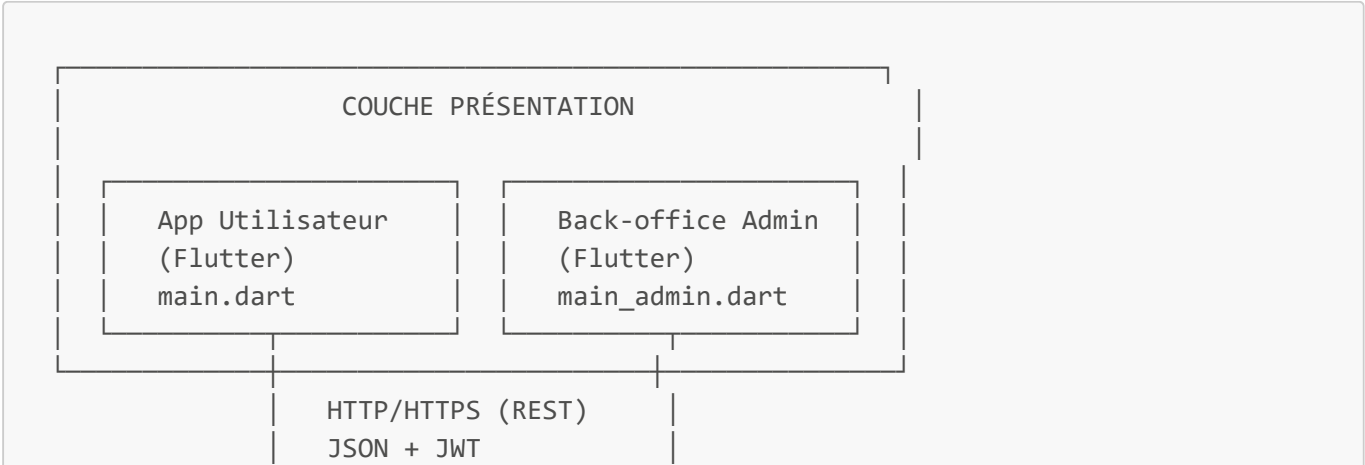
1.4 Fonctionnalités principales

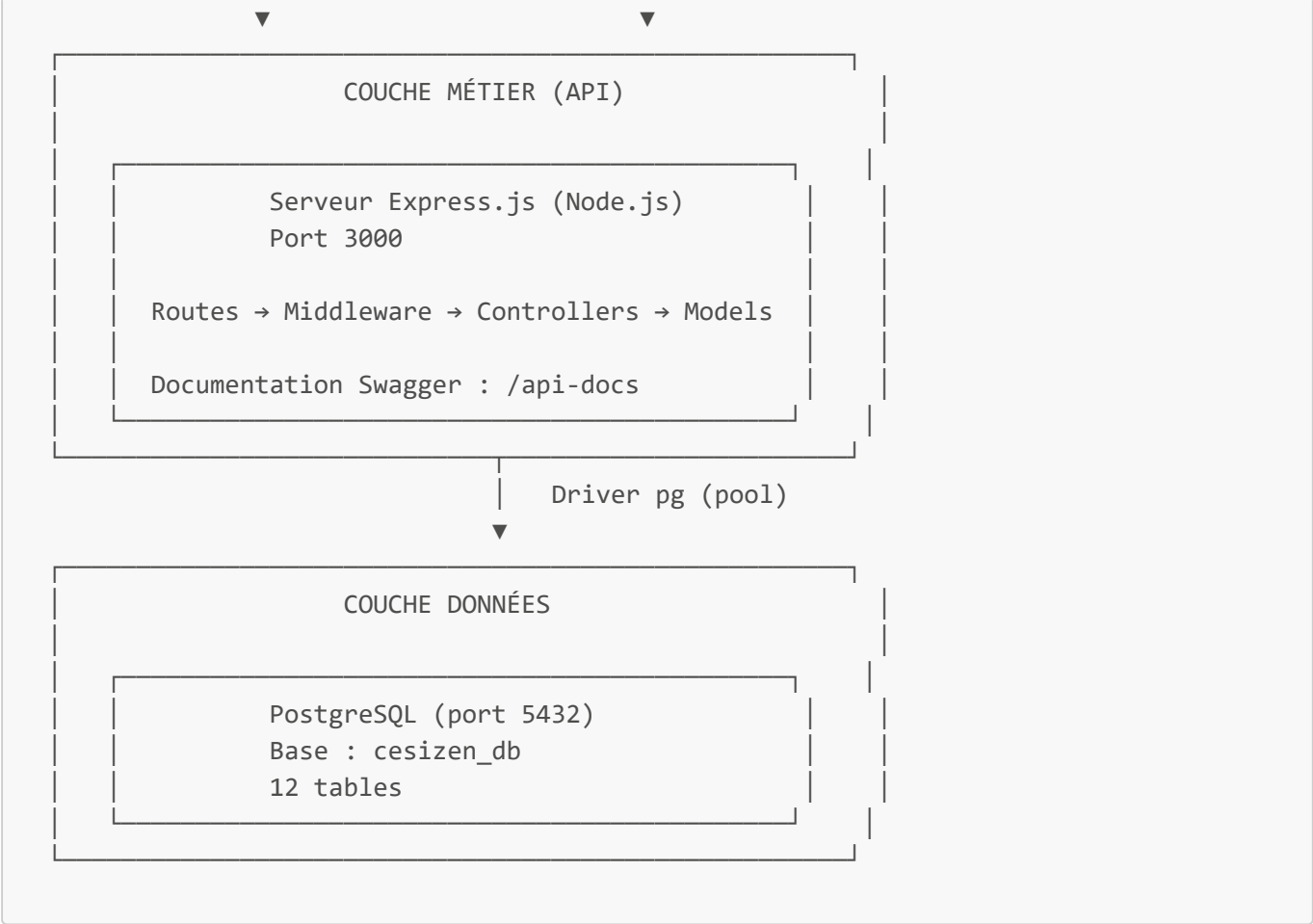
Fonctionnalité	Utilisateur	Administrateur
Inscription / Connexion	✓	✓ (interface dédiée)
Questionnaire de stress	✓	—
Tracker d'émotions	✓	—
Consultation des activités	✓	—
Gestion des favoris	✓	—
Gestion des activités (CRUD)	—	✓
Gestion des utilisateurs	—	✓
Activation/désactivation comptes	—	✓

2. Architecture générale du système

2.1 Vue d'ensemble

CesiZen repose sur une architecture **trois tiers** classique :





2.2 Flux de communication

Les échanges entre le client Flutter et l'API suivent systématiquement le schéma suivant :

- 1. Le client envoie une requête HTTP avec un corps JSON
- 2. Le middleware d'authentification valide le token JWT (si route protégée)
- 3. Le contrôleur exécute la logique métier via le modèle
- 4. Le modèle exécute la requête SQL via le pool de connexions
- 5. La réponse JSON est renvoyée au client avec le code HTTP approprié

2.3 Environnements cibles

Environnement	URL API	Usage
Développement web	http://localhost:3000/api	Tests navigateur
Émulateur Android	http://10.0.2.2:3000/api	Tests émulateur
Production	À configurer	Déploiement

3. Comparatif des solutions techniques

Avant de choisir la stack technologique, plusieurs alternatives ont été évaluées selon des critères objectifs. Voici l'analyse comparative pour chacune des trois couches principales.

3.1 Comparatif des architectures backend

Trois architectures backend ont été considérées :

Critère	Architecture 1 Node.js + Express	Architecture 2 Spring Boot (Java)	Architecture 3 Django REST (Python)
Performance (req/s)	Très élevée (event loop non-bloquante)	Élevée (JVM optimisée)	Moyenne (GIL Python)
Courbe d'apprentissage	Faible (JS connu de l'équipe)	Élevée (Java, annotations Spring)	Moyenne (Python accessible)
Écosystème npm/pip	Très riche (2M+ packages)	Riche (Maven/Gradle)	Riche (PyPI)
Scalabilité	Excellente (horizontale via clusters)	Excellente (JVM, microservices)	Bonne (Celery, Gunicorn)
Temps de démarrage	Rapide (< 1s)	Lent (JVM warmup, 5-15s)	Moyen (2-5s)
Intégration PostgreSQL	Native (driver pg)	Native (JDBC, Spring Data JPA)	Native (psycopg2, ORM Django)
Génération doc API	Facile (swagger-jsdoc)	Automatique (SpringDoc)	Automatique (drf-spectacular)
Gestion JWT	Simple (jsonwebtoken)	Spring Security (complexe)	Bibliothèque tierce (DRF JWT)
Coût d'hébergement	Faible (Node léger)	Élevé (RAM JVM > 256 MB)	Faible

Notation (1-5) :

Critère	Node.js + Express	Spring Boot	Django REST
Performance I/O	5	4	3
Facilité de développement	5	2	4
Maintenance long terme	4	5	4
Rapidité de prototypage	5	2	4
Consommation ressources	5	2	4
Total	24/25	15/25	19/25

3.2 Comparatif des solutions mobile

Trois approches de développement mobile ont été étudiées :

Critère	Solution 1 Flutter (Dart)	Solution 2 React Native (JS)	Solution 3 Natif (Swift/Kotlin)
Plateformes cibles	iOS, Android, Web, Desktop	iOS, Android	iOS OU Android (séparé)
Performance UI	Excellente (moteur Skia/Impeller)	Bonne (bridge JS natif)	Optimale
Codebase unique	✓ (100% partagé)	✓ (90% partagé)	X (deux codebases)
Hot reload	✓	✓	Partiel
Widgets personnalisés	Très facile (Widgets composables)	Possible (composants RN)	Complexe
Taille APK	~15-20 MB	~25-30 MB	~5-10 MB
Communauté	Grande (Google)	Très grande (Meta/Microsoft)	Grande
Tests unitaires	Excellent (flutter_test, mockito)	Bon (Jest, React Testing Library)	Bon
Intégration sécurité	flutter_secure_storage natif	react-native-keychain	Keychain/KeyStore natif
Courbe d'apprentissage	Moyenne (Dart à apprendre)	Faible (JS/TS connu)	Élevée (deux langages)

Notation (1-5) :

Critère	Flutter	React Native	Natif
Multiplateforme	5	4	2
Performance	5	3	5
Productivité	4	4	2
Expérience UI	5	3	5
Écosystème packages	4	5	4
Total	23/25	19/25	18/25

3.3 Comparatif des systèmes de gestion de base de données

Critère	PostgreSQL	MySQL	MongoDB
Type	Relationnel (SQL)	Relationnel (SQL)	NoSQL (Document)
Transactions ACID	✓ (complet)	✓ (InnoDB)	Partiel (depuis v4.0)
Clés étrangères	✓ natif	✓ natif	X (relations simulées)

Critère	PostgreSQL	MySQL	MongoDB
Requêtes complexes (JOIN)	Excellent	Bon	Difficile
JSON natif	✓ (jsonb)	Partiel	✓ (natif)
Performances	Très bonnes	Bonnes	Excellentes (lecture)
Cohérence des données	Forte	Forte	Éventuelle
Agrégations	SQL standard	SQL standard	Pipeline d'agrégation
Licence	Open Source (MIT)	GPL (Oracle)	SSPL (restrictif)
Adapté à notre schéma	Oui (données fortement liées)	Oui	Non (schéma relationnel)

Notation (1-5) :

Critère	PostgreSQL	MySQL	MongoDB
Intégrité des données	5	4	2
Flexibilité schéma	4	3	5
Performance requêtes jointes	5	4	2
Facilité d'administration	4	4	4
Licence et coût	5	3	3
Total	23/25	18/25	16/25

4. Pertinence de la solution retenue

4.1 Stack technologique choisie

La solution retenue est :

- **Backend** : Node.js 20 LTS + Express.js 5
- **Base de données** : PostgreSQL 14+
- **Application mobile** : Flutter 3.10+ (Dart)
- **Authentification** : JSON Web Tokens (JWT)
- **Documentation API** : OpenAPI 3.0 / Swagger UI

4.2 Justification des choix

Cohérence technologique (JavaScript full-stack)

Le choix de Node.js pour le backend et Flutter pour le mobile permet à l'équipe de maintenir deux codebases dans des langages proches (JavaScript/Dart), réduisant la charge cognitive et accélérant le développement. La syntaxe asynchrone (`async/await`) est commune aux deux environnements.

Performance et scalabilité

Node.js utilise un modèle d'entrées/sorties non-bloquant basé sur une boucle d'événements (event loop). Pour une API REST dont les opérations sont principalement des lectures/écritures en base de données, ce modèle est optimal : le serveur ne bloque pas en attendant les réponses PostgreSQL, permettant de traiter de nombreuses requêtes concurrentes avec un minimum de ressources.

Les benchmarks publiés par TechEmpower (Round 22) montrent que les frameworks Node.js (Fastify, Express) surpassent largement Spring Boot en termes de requêtes par seconde pour les charges I/O-bound, avec une consommation mémoire 3 à 5 fois moindre.

Maintenabilité

L'architecture MVC (Model-View-Controller) adoptée sépare clairement :

- La définition des routes (routes/)
- La logique métier (controller/)
- L'accès aux données (model/)
- Les middlewares transverses (middlewares/)

Cette séparation facilite les évolutions futures : ajouter un endpoint ne nécessite que d'ajouter une route et un contrôleur sans impacter les autres couches.

Fiabilité des données

PostgreSQL est le seul SGBD open source offrant une conformité ACID complète avec support natif des contraintes d'intégrité référentielle. Le modèle de données de CesiZen est fortement relationnel (utilisateurs → diagnostics → questions, activités → catégories, etc.), ce qui justifie pleinement le choix d'un SGBD relationnel plutôt que NoSQL.

Expérience utilisateur Flutter

Flutter compile en code natif ARM via son moteur de rendu Impeller (depuis Flutter 3.10), offrant des animations à 60/120 fps sans pont JavaScript. Le hot reload accélère considérablement le cycle de développement. La capacité à cibler iOS et Android depuis un seul codebase divise par deux le travail de développement UI.

Sécurité

Les JWT permettent une authentification stateless scalable. Les tokens utilisateurs expirent après 24h, ceux des administrateurs après 8h (durée plus courte car accès plus sensible). Les mots de passe sont hashés avec bcrypt (coût 10), algorithme spécifiquement conçu pour être lent et résistant aux attaques par force brute.

4.3 Compromis acceptés

Compromis	Explication
Node.js single-threaded	Pour des traitements CPU intensifs, Node.js serait moins adapté. Dans notre cas, les calculs (score de stress) sont simples et non-bloquants.
Dart (Dart vs JS/TS)	Dart est moins connu que TypeScript, mais sa syntaxe proche de Java/C# est rapidement assimilable.

Compromis	Explication
ORM vs SQL direct	L'utilisation de requêtes SQL directes (sans ORM) offre plus de contrôle mais exige plus de rigueur dans la gestion des requêtes paramétrées.

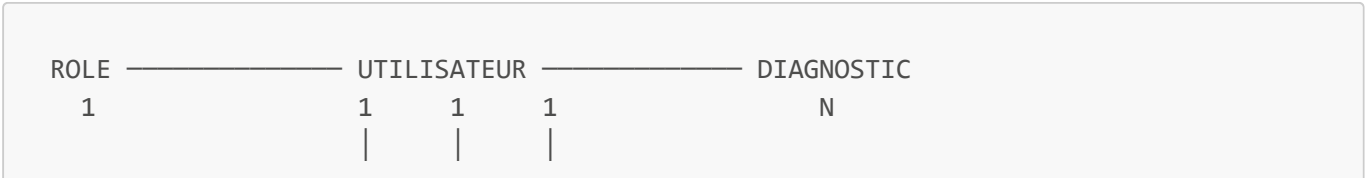
5. Modèle Logique de Données (MLD)

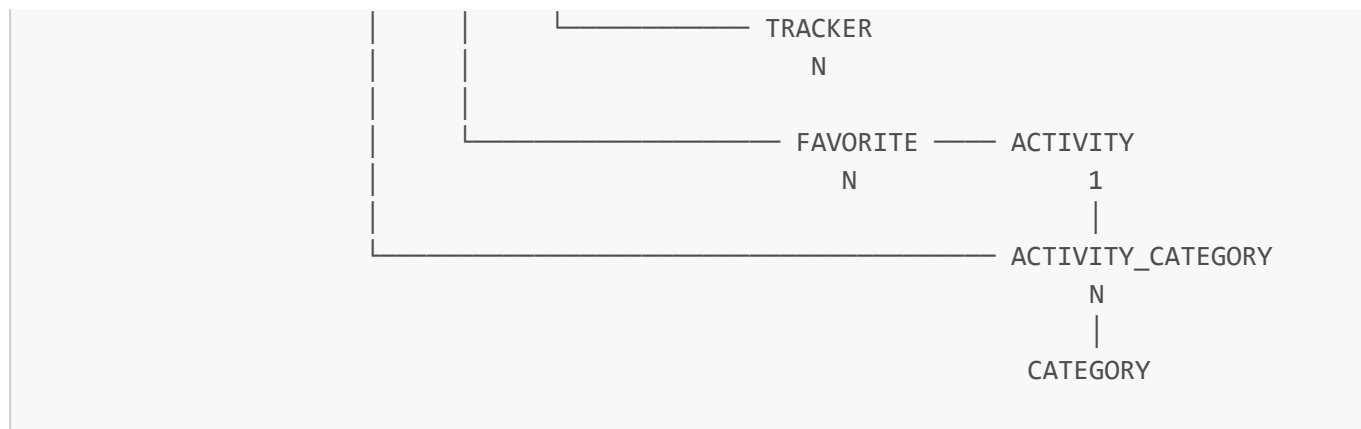
5.1 Schéma MLD

Le MLD de CesiZen comprend 12 tables organisées autour de l'entité centrale **utilisateur**.

ROLE (id_role [PK], nom_role)
UTILISATEUR (id_utilisateur [PK], pseudo, email [UNIQUE], mot_de_passe, id_role [FK→ROLE], est_actif)
QUESTION (id_question [PK], contenu, val_score)
EMOTION (id_emotion [PK], nom_emotion, emotion_primaire)
DIAGNOSTIC (id_diagnostic [PK], date_diag, score, nv_stress, id_utilisateur [FK→UTILISATEUR], id_question [FK→QUESTION])
TRACKER (id_entree [PK], commentaire, date_entree, id_emotion [FK→EMOTION], id_utilisateur [FK→UTILISATEUR])
ACTIVITY (id_activity [PK], title, content, activity_url, image_url, short_description, activity_date, est_active, id_utilisateur [FK→UTILISATEUR])
CATEGORY (id_category [PK], title, icon_name, color_code)
ACTIVITY_CATEGORY (id_activity [FK→ACTIVITY], id_category [FK→CATEGORY]) [PK composée : (id_activity, id_category)]
FAVORITE (id_utilisateur [FK→UTILISATEUR], id_activity [FK→ACTIVITY]) [PK composée : (id_utilisateur, id_activity)]
ARTICLE (id_article [PK], title, content, article_type, published, id_utilisateur [FK→UTILISATEUR])
TYPE_DICTIONNARY (id_type [PK], nom_type, id_activity [FK→ACTIVITY], id_article [FK→ARTICLE])

5.2 Diagramme des relations





5.3 Description détaillée des tables

Table **role**

Référentiel des rôles applicatifs. Deux valeurs fixes : **admin** (id=1) et **user** (id=2).

Colonne	Type	Contrainte	Description
id_role	SERIAL	PK	Identifiant auto-incrémenté
nom_role	VARCHAR(100)	NOT NULL	Libellé du rôle

Table **utilisateur**

Table centrale de l'application. Stocke les comptes utilisateurs et administrateurs.

Colonne	Type	Contrainte	Description
id_utilisateur	SERIAL	PK	Identifiant unique
pseudo	VARCHAR(100)	NOT NULL	Nom d'affichage
email	VARCHAR(100)	NOT NULL, UNIQUE	Adresse email (identifiant de connexion)
mot_de_passe	VARCHAR(100)	NOT NULL	Hash bcrypt du mot de passe
id_role	INT	FK→role	Rôle de l'utilisateur (1=admin, 2=user)
est_actif	BOOLEAN	—	Statut du compte (désactivable par admin)

Table **question**

Questions du questionnaire Holmes-Rahe pour le diagnostic de stress.

Colonne	Type	Contrainte	Description
id_question	SERIAL	PK	Identifiant
contenu	TEXT	NOT NULL	Texte de la question/événement de vie
val_score	INT	—	Points associés à l'événement (11 à 100)

Table *emotion*

Référentiel des émotions pour le tracker. Organisé en émotions primaires et secondaires.

Colonne	Type	Contrainte	Description
id_emotion	SERIAL	PK	Identifiant
nom_emotion	VARCHAR(100)	NOT NULL	Libellé de l'émotion (ex: "Anxieux")
emotion_primare	VARCHAR(100)	—	Émotion primaire parente (ex: "Peur")

Données de référence :

- Heureux → Joie | Serein → Joie | Anxieux → Peur | Stressé → Peur | Triste → Tristesse | En colère → Colère | Fatigué → Fatigue

Table *diagnostic*

Enregistre les résultats des diagnostics de stress passés par les utilisateurs.

Colonne	Type	Contrainte	Description
id_diagnostic	SERIAL	PK	Identifiant
date_diag	DATE	—	Date du diagnostic
score	INT	—	Score total calculé
nv_stress	INT	—	Niveau de stress (1=faible, 2=modéré, 3=élevé)
id_utilisateur	INT	FK→utilisateur	Utilisateur ayant passé le test
id_question	INT	FK→question	Référence question (contexte)

Règle métier de calcul du niveau de stress :

- Niveau 1 : score < 100 (faible)
- Niveau 2 : score entre 100 et 300 (modéré)
- Niveau 3 : score > 300 (élevé)

Table *tracker*

Journal quotidien des émotions de l'utilisateur.

Colonne	Type	Contrainte	Description
id_entree	SERIAL	PK	Identifiant
commentaire	TEXT	—	Note libre associée
date_entree	DATE	—	Date de l'entrée
id_emotion	INT	FK→emotion	Émotion ressentie

Colonne	Type	Contrainte	Description
id_utilisateur	INT	FK→utilisateur	Auteur de l'entrée

Table **activity**

Catalogue des activités bien-être proposées aux utilisateurs.

Colonne	Type	Contrainte	Description
id_activity	SERIAL	PK	Identifiant
title	VARCHAR(100)	NOT NULL	Titre de l'activité
content	TEXT	—	Description complète
short_description	TEXT	—	Description courte (card UI)
activity_url	TEXT	—	Lien vidéo (YouTube)
image_url	TEXT	—	URL de l'image d'illustration
activity_date	DATE	—	Date de création
est_active	BOOLEAN	—	Visibilité pour les utilisateurs
id_utilisateur	INT	FK→utilisateur	Administrateur créateur

Table **category**

Référentiel des catégories d'activités avec métadonnées visuelles.

Colonne	Type	Contrainte	Description
id_category	SERIAL	PK	Identifiant
title	VARCHAR(50)	NOT NULL	Libellé de la catégorie
icon_name	VARCHAR(50)	—	Nom de l'icône Material Design
color_code	VARCHAR(10)	—	Code couleur hexadécimal

Table **activity_category**

Table de liaison permettant l'association multiple activité ↔ catégorie.

Colonne	Type	Contrainte
id_activity	INT	FK→activity, PK composée
id_category	INT	FK→category, PK composée

Table **favorite**

Favoris des utilisateurs. Clé primaire composée pour garantir l'unicité.

Colonne	Type	Contrainte
id_utilisateur	INT	FK→utilisateur, PK composée
id_activity	INT	FK→activity, PK composée

Table **article**

Articles informatifs liés au bien-être (fonctionnalité en cours de développement).

Colonne	Type	Contrainte	Description
id_article	SERIAL	PK	Identifiant
title	VARCHAR(200)	NOT NULL	Titre de l'article
content	TEXT	—	Contenu complet
article_type	VARCHAR(100)	—	Type d'article
published	DATE	—	Date de publication
id_utilisateur	INT	FK→utilisateur	Auteur

Table **type_dictionnaire**

Table de classification des types de contenus.

Colonne	Type	Contrainte	Description
id_type	SERIAL	PK	Identifiant
nom_type	VARCHAR(100)	NOT NULL	Libellé du type
id_activity	INT	FK→activity	Activité associée
id_article	INT	FK→article	Article associé

6. Architecture backend détaillée

6.1 Structure des fichiers

```
backend/  
├─ app.js                # Point d'entrée, configuration Express +  
Swagger  
├─ package.json          # Dépendances Node.js  
├─ jest.config.js        # Configuration tests Jest  
├─ .env                  # Variables d'environnement (non versionné)  
├─  
├─ src/  
│   └─ config/  
│       └─ db.js         # Pool de connexions PostgreSQL  
│  
└─
```

```

├── routes/
│   ├── authRoute.js          # Routes authentication
│   ├── activityRoute.js      # Routes activités
│   ├── categoryRoute.js      # Routes catégories
│   ├── favoriteRoute.js      # Routes favoris
│   ├── diagnosticRoute.js    # Routes diagnostic
│   └── userRoute.js          # Routes gestion utilisateurs
├── controller/
│   ├── authController.js     # Logique login/register/adminLogin
│   ├── activityController.js # CRUD activités
│   ├── categoryController.js # Lecture catégories
│   ├── favoriteController.js # Toggle/check favoris
│   ├── diagnosticController.js # Questionnaire + sauvegarde
│   └── userController.js     # Gestion utilisateurs (admin)
├── model/
│   ├── authModel.js          # Requêtes SQL utilisateur
│   ├── activityModel.js      # Requêtes SQL activités
│   └── categoryModel.js      # Requêtes SQL catégories
├── middlewares/
│   └── authMiddleware.js     # Validation JWT
└── tests/
    ├── auth.test.js          # Tests authentication
    ├── activity.test.js      # Tests activités
    ├── favorite.test.js      # Tests favoris
    ├── protected_routes.test.js # Tests middleware auth
    └── teardown.js           # Nettoyage après tests

```

6.2 Configuration de la base de données (src/config/db.js)

La connexion PostgreSQL utilise un **pool de connexions** (pg.Pool) pour optimiser les performances. Le pool gère automatiquement la réutilisation des connexions TCP, évitant le coût d'établissement d'une nouvelle connexion à chaque requête.

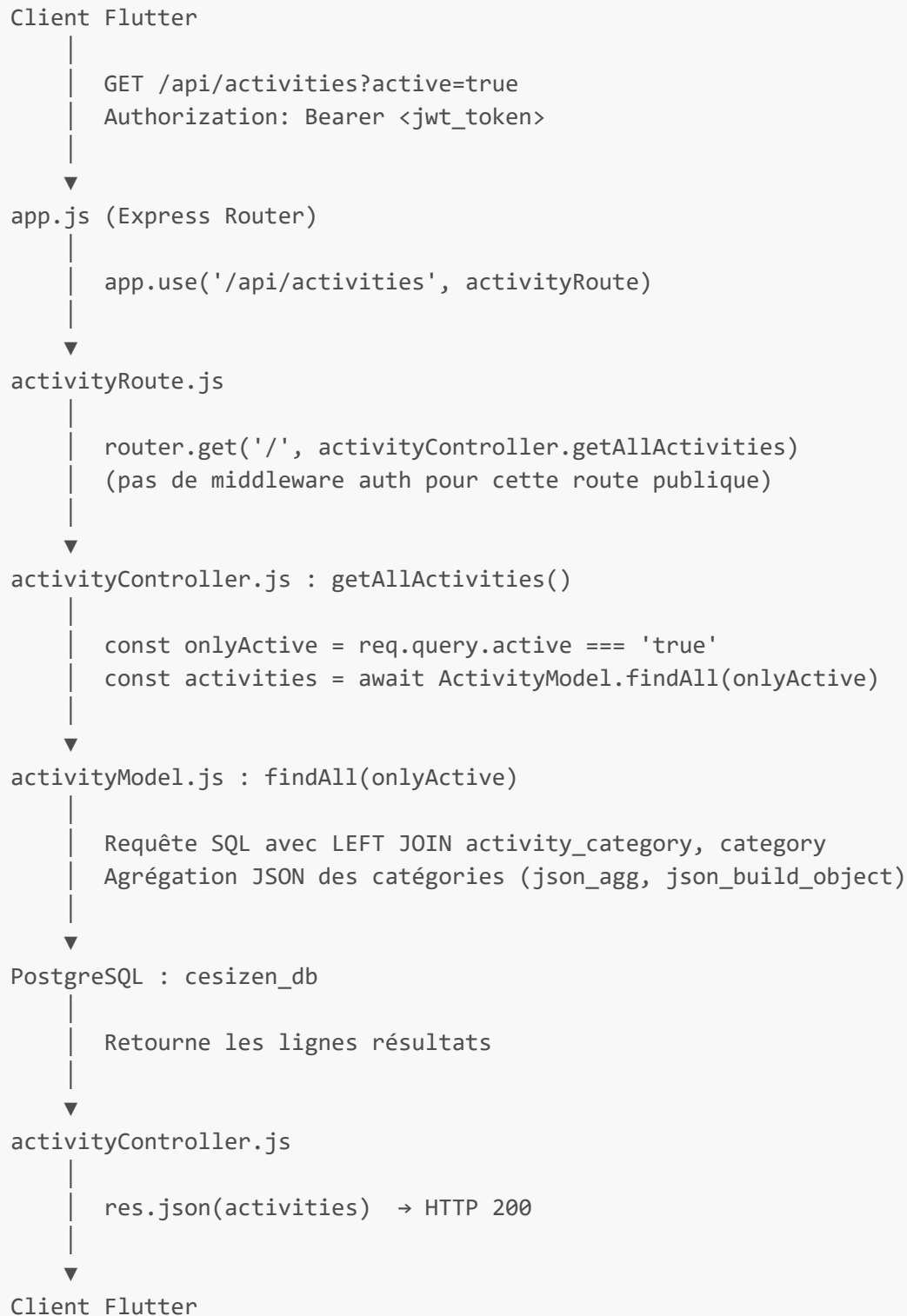
```

// Variables d'environnement requises
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=root
DB_NAME=cesizen_db

```

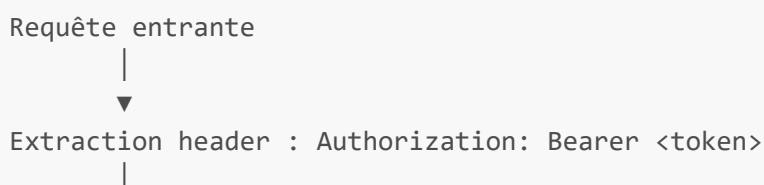
6.3 Flux d'une requête type

Voici le flux d'une requête authentifiée (**GET /api/activities**) :



6.4 Middleware d'authentification

Le middleware JWT intercepte chaque requête sur les routes protégées :



```
Token présent ?
┌ NO ──> 401 "Accès refusé"
└ OUI
  │
  ▼
  jwt.verify(token, JWT_SECRET)
  │
  Token valide ?
  ┌ NO ──> 403 "Token invalide ou expiré"
  └ OUI
    │
    ▼
    req.user = { id, role } → next()
```

7. Documentation de l'API REST

L'API suit les conventions REST. Tous les échanges sont en JSON. La documentation interactive Swagger est disponible sur <http://localhost:3000/api-docs>.

7.1 Base URL

```
http://localhost:3000/api
```

7.2 Authentification

Les endpoints protégés nécessitent un header :

```
Authorization: Bearer <token_jwt>
```

7.3 Endpoints Authentication ([/api/auth](#))

POST [/api/auth/register](#) — Inscription

Corps de la requête :

```
{
  "pseudo": "JohnDoe",
  "email": "john@example.com",
  "password": "motdepasse123"
}
```

Réponses :

- 200 OK : { "message": "Utilisateur créé", "user": {...} }
- 400 Bad Request : Champs manquants ou email déjà utilisé
- 500 Internal Server Error : Erreur serveur

POST /api/auth/login — Connexion utilisateur

Corps de la requête :

```
{  
  "email": "john@example.com",  
  "password": "motdepasse123"  
}
```

Réponses :

- 200 OK : { "token": "<jwt>", "user": { "id": 1, "role": 2, "pseudo": "JohnDoe" } }
- 400 Bad Request : Champs manquants
- 401 Unauthorized : Identifiants incorrects
- 403 Forbidden : Compte désactivé
- 500 Internal Server Error : Erreur serveur

POST /api/auth/admin/login — Connexion administrateur

Corps de la requête :

```
{ "email": "admin@cesizen.fr", "password": "adminpass" }
```

Réponses :

- 200 OK : { "token": "<jwt_admin_8h>" }
- 401 Unauthorized : Mot de passe incorrect
- 403 Forbidden : Non admin ou compte inexistant

GET /api/auth/profil — Profil connecté (*protégé*)

Réponses :

- 200 OK : Données du profil utilisateur
- 401 Unauthorized : Token absent
- 403 Forbidden : Token invalide

7.4 Endpoints Activités (/api/activities)

GET /api/activities — Liste des activités

Query params optionnel : ?active=true (uniquement les activités visibles)

Réponse 200 :

```
[
  {
    "id_activity": 4,
    "title": "Détente en 5 minutes",
    "content": "Description complète...",
    "short_description": "Exercices de yoga",
    "activity_url": "https://www.youtube.com/watch?v=...",
    "image_url": "https://...",
    "activity_date": "2024-07-05",
    "est_active": true,
    "categories": [{ "id_category": 1, "title": "Yoga", "icon_name": "...",
"color_code": "#..." }]
  }
]
```

POST /api/activities — Créer une activité (protégé)**Corps :**

```
{
  "title": "Titre",
  "content": "Description longue",
  "short_description": "Résumé",
  "activity_url": "https://youtube.com/...",
  "image_url": "https://...",
  "id_utilisateur": 1,
  "id_category": 2
}
```

Réponse 201 : { "id_activity": 5, "message": "Activité créée avec sa catégorie" }

Note : La création d'activité utilise une **transaction SQL** (BEGIN/COMMIT/ROLLBACK) pour garantir l'atomicité de l'insertion dans **activity** ET **activity_category**.

PUT /api/activities/:id — Modifier une activité (protégé)**Réponse 200 :** { "message": "Activité modifiée avec succès" }**PATCH /api/activities/:id/status — Activer/désactiver (protégé)****Corps :** { "est_active": false } **Réponse 200 :** { "message": "Statut de l'activité mis à jour : false" }**DELETE /api/activities/:id — Supprimer (protégé)**

- 200 OK : Supprimée

- 404 Not Found : ID inexistant

7.5 Endpoints Catégories (/api/categories)

GET /api/categories — Liste des catégories

Réponse 200 : [{ "id_category": 1, "title": "Yoga", "icon_name": "yoga", "color_code": "#6BBCB5" }]

7.6 Endpoints Diagnostic (/api/diagnostic)

GET /api/diagnostic/questions — Toutes les questions

Réponse 200 : Tableau des 43 questions Holmes-Rahe avec leur score.

POST /api/diagnostic/save — Sauvegarder un résultat

Corps : { "id_utilisateur": 3, "score": 250 }

Le backend calcule automatiquement le `nv_stress` à partir du score :

```
const nv_stress = score < 100 ? 1 : score <= 300 ? 2 : 3;
```

Réponse 201 : { "message": "Diagnostic enregistré" }

PUT /api/diagnostic/config/:id — Modifier la configuration (admin)

Corps : { "titre": "Nouveau titre", "description": "Nouveau texte" }

7.7 Endpoints Favoris (/api/favorite)

POST /api/favorite/toggle — Ajouter/retirer un favori (protégé)

Corps : { "id_utilisateur": 3, "id_activity": 4 }

- Si le favori existe : suppression → { "favorited": false }
- Si absent : ajout → { "favorited": true }

GET /api/favorite/check — Vérifier un favori (protégé)

Query params : ?id_utilisateur=3&id_activity=4 **Réponse :** { "isFavorited": true }

7.8 Endpoints Utilisateurs (/api/users) (admin)

GET /api/users — Liste tous les utilisateurs

Réponse 200 : [{ "id_utilisateur": 2, "pseudo": "JohnDoe", "email": "...", "id_role": 2, "est_actif": true }]

PUT /api/users/:id/role — Changer le rôle**Corps :** { "id_role": 1 }**PATCH /api/users/:id/status — Activer/désactiver un compte****Corps :** { "est_actif": false }

8. Architecture mobile Flutter

8.1 Structure du projet

```

mobile_app/lib/
├── main.dart                # Point d'entrée – App Utilisateur
├── main_admin.dart          # Point d'entrée – Back-office Admin
├── config.dart              # URL API selon plateforme
├──
├── models/
│   ├── activity.dart        # Modèle Activité + désérialisation JSON
│   ├── activity_category.dart # Modèle Catégorie
│   └── question.dart        # Modèle Question
├──
├── services/
│   ├── auth_service.dart    # Login, register, logout, token
│   ├── activity_service.dart # Récupération activités
│   ├── category_service.dart # Récupération catégories
│   ├── diagnostic_service.dart # Questions + sauvegarde résultat
│   └── favorite_service.dart # Toggle/check favoris
├──
├── views/
│   ├── login_screen.dart    # Écran de connexion
│   ├── register_screen.dart # Écran d'inscription
│   ├── home_screen.dart     # Accueil + navigation bas
│   ├── activity_screen.dart  # Liste des activités
│   ├── activity_detail_screen.dart # Détail + lecteur YouTube
│   ├── category_screen.dart  # Navigation par catégorie
│   ├── diagnostic_start_screen.dart # Intro diagnostic
│   ├── diagnostic_questionnaire_screen.dart # Questionnaire
│   ├── diagnostic_result_screen.dart # Résultat stress
│   ├── profile_screen.dart   # Profil utilisateur
│   ├── profile_settings_screen.dart # Paramètres profil
│   └── placeholder_screen.dart # Écrans en construction
├──
├── ui/
│   ├── theme.dart           # Système de design (couleurs, typographie)
│   └── widgets/
│       ├── activity_card.dart # Carte activité
│       ├── category_card.dart # Carte catégorie
│       ├── category_pill.dart # Filtre catégorie
│       ├── form_input.dart    # Champ texte avec icône
│       └── diag_input.dart    # Input questionnaire

```

```

├── boutons.dart          # Composants boutons
├── navigation_bar.dart   # Barre navigation bas
├── top_bar.dart          # Barre supérieure
├── top_bar_rounded.dart  # Barre supérieure arrondie
├── page_layout.dart      # Conteneur de page
├── widgets.dart          # Exports partagés
├── backoffice/
│   ├── admin_auth_wrapper.dart # Vérification auth admin
│   ├── backoffice_layout.dart  # Layout back-office
│   ├── admin_login_screen.dart # Connexion admin
│   ├── activity_management_page.dart # Gestion activités
│   ├── user_management_page.dart # Gestion utilisateurs
│   ├── services/
│   │   ├── admin_auth_service.dart # JWT admin
│   │   ├── admin_activity_service.dart # CRUD activités
│   │   └── admin_user_service.dart # Gestion utilisateurs
│   └── models/
│       ├── admin_activity_model.dart # Modèle activité complet
│       ├── admin_category_model.dart # Modèle catégorie
│       └── user_model.dart # Modèle utilisateur admin

```

8.2 Système de design (Design System)

Le système de design CesiZen définit une identité visuelle cohérente.

Palette de couleurs :

Nom	Code hexadécimal	Usage
Sky Blue	#6BBCD0	Accents secondaires
Tropical Teal	#6BBCB5	Couleur primaire, CTA
Dark Cyan	#528E8A	Éléments actifs, focus
Soft Peach	#F2D492	Alertes, highlights
Dry Sage	#B8B08D	Textes secondaires

Typographie :

- Titres : **Merriweather Sans** (via Google Fonts)
- Corps : **Lato** (via Google Fonts)
- Hiérarchie : H1 → H5 + regular / big / little text

8.3 Gestion de l'authentification mobile

L'**AuthService** implémente le pattern d'**injection de dépendances** pour faciliter les tests unitaires :

```
// Injection des dépendances pour les tests
AuthService({http.Client? client, FlutterSecureStorage? storage});
```

Le token JWT est stocké de manière sécurisée via `flutter_secure_storage` qui utilise :

- **Android** : Android Keystore (EncryptedSharedPreferences)
- **iOS** : Keychain

Données persistées :

- `jwt_token` : Token d'authentification
- `user_id` : Identifiant utilisateur
- `pseudo` : Pseudo pour affichage

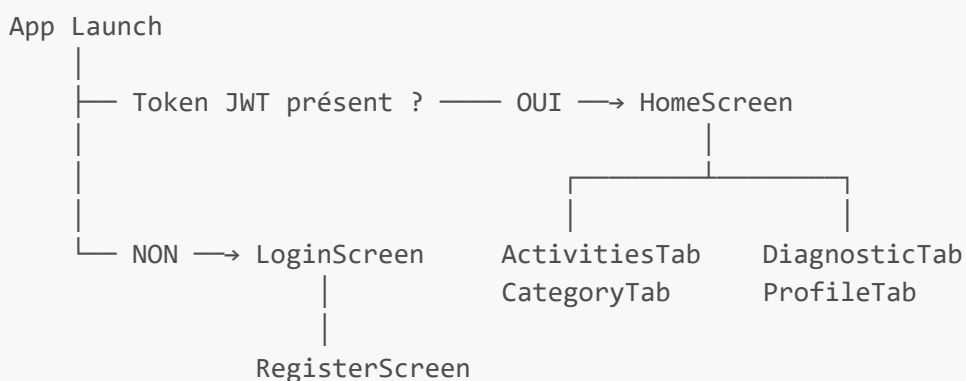
8.4 Architecture des services Flutter

Chaque service suit le même pattern :

```
class ActivityService {
  final http.Client client;
  final String baseUrl;

  // 1. Récupération du token
  // 2. Construction de la requête HTTP
  // 3. Désérialisation JSON → modèle Dart
  // 4. Gestion des erreurs (throw Exception)
}
```

8.5 Flux de navigation utilisateur



8.6 Application back-office administrateur

Le back-office est une application Flutter indépendante (point d'entrée `main_admin.dart`) qui expose uniquement les fonctionnalités de gestion :

1. **Authentification admin** via `/api/auth/admin/login` (token 8h)
2. **Gestion des activités** : Création, modification, activation/désactivation, suppression
3. **Gestion des utilisateurs** : Liste, changement de rôle, activation/désactivation des comptes

Le composant `AdminAuthWrapper` vérifie la présence du token admin au lancement et redirige vers la page de connexion si absent ou expiré.

9. Sécurité

9.1 Authentification et autorisation

Hashage des mots de passe (bcrypt)

Lors de l'inscription, le mot de passe est hashé avec bcrypt (coût 10) :

```
const hashedPassword = await bcrypt.hash(password, 10);
```

bcrypt est un algorithme intentionnellement lent, conçu pour résister aux attaques par dictionnaire et force brute. Le coût 10 correspond à $2^{10} = 1024$ itérations.

JSON Web Tokens (JWT)

Le payload du token contient :

```
{ "id": 3, "role": 2, "iat": 1748000000, "exp": 1748086400 }
```

Type de token	Expiration	Payload
Utilisateur	24 heures	<code>{ id, role }</code>
Administrateur	8 heures	<code>{ userId, role: 'admin' }</code>

Protection des routes

Le middleware `authMiddleware` est appliqué sur toutes les routes sensibles. Il vérifie :

1. Présence du header `Authorization: Bearer <token>`
2. Validité de la signature JWT (secret stocké côté serveur)
3. Non-expiration du token

9.2 Gestion des comptes désactivés

Un utilisateur dont le compte est désactivé (`est_actif = false`) reçoit une erreur `403 Forbidden` même avec des identifiants corrects. Cela permet aux administrateurs de suspendre l'accès sans supprimer les données.

9.3 Stockage sécurisé mobile

Les tokens sont stockés via `flutter_secure_storage` (chiffrement AES côté Android, Keychain côté iOS). Aucune donnée sensible n'est écrite dans les SharedPreferences non chiffrées.

9.4 Protection contre les injections SQL

Toutes les requêtes utilisent des **requêtes paramétrées** avec le driver `pg` :

```
pool.query('SELECT * FROM utilisateur WHERE email = $1', [email])
```

Les paramètres ne sont jamais interpolés directement dans la chaîne SQL, ce qui prévient les injections SQL.

9.5 CORS

La configuration CORS (`cors()`) est activée pour permettre les requêtes cross-origin depuis le client Flutter. En production, ce middleware devrait être restreint aux domaines autorisés.

9.6 Variables d'environnement

Les secrets (clé JWT, credentials base de données) ne sont jamais écrits en dur dans le code source. Ils sont chargés depuis un fichier `.env` via `dotenv`, qui doit être exclu du versionnement (`.gitignore`).

10. Stratégie de tests

10.1 Tests backend (Jest + Supertest)

Le backend dispose d'une suite de tests d'intégration utilisant Jest et Supertest. Les tests sont exécutés sur une base de données de test dédiée.

Configuration (`jest.config.js`) :

```
{
  testEnvironment: 'node',
  globalTeardown: './tests/teardown.js'
}
```

Scripts npm disponibles :

```
npm test           # Lance tous les tests
npm run test:coverage # Lance avec rapport de couverture
npm run test:json    # Exporte les résultats en JSON
```

Fichiers de tests :

Fichier	Couverture
<code>auth.test.js</code>	Login (succès/échec), Register (succès/email dupliqué), validation champs
<code>activity.test.js</code>	GET liste, POST création, PUT modification, PATCH status, DELETE
<code>favorite.test.js</code>	Toggle (add/remove), Check favori
<code>protected_routes.test.js</code>	Routes protégées sans token, avec token invalide, avec token expiré
<code>teardown.js</code>	Nettoyage des données de test, fermeture pool DB

Exemple de test d'authentification :

```
describe('POST /api/auth/login', () => {
  it('retourne 200 et un token pour des identifiants valides', async () => {
    const res = await request(app)
      .post('/api/auth/login')
      .send({ email: 'test@example.com', password: 'password' });
    expect(res.status).toBe(200);
    expect(res.body).toHaveProperty('token');
  });

  it('retourne 401 pour un mauvais mot de passe', async () => {
    const res = await request(app)
      .post('/api/auth/login')
      .send({ email: 'test@example.com', password: 'wrong' });
    expect(res.status).toBe(401);
  });
});
```

10.2 Tests mobile (Flutter Test + Mockito)

Les services Flutter sont testés en isolation grâce à des mocks générés par `mockito` + `build_runner`.

Fichiers de tests :

Fichier	Couverture
<code>auth_service_test.dart</code>	Login succès/échec, Register, déduplication email, erreur réseau, persistance token
<code>favorite_service_test.dart</code>	Toggle favori, vérification état, gestion erreurs HTTP

Pattern de test :

```
// Génération automatique des mocks
@GenerateMocks([http.Client])
void main() {
  late MockClient mockClient;
  late FakeFlutterSecureStorage fakeStorage;
```



```
setUp(() {
  mockClient = MockClient();
  fakeStorage = FakeFlutterSecureStorage();
});

test('login retourne true et stocke le token', () async {
  when(mockClient.post(...)).thenAnswer((_) async =>
    http.Response('{"token":"abc123","user":{"id":1,...}}', 200));

  final service = AuthService(client: mockClient, storage: fakeStorage);
  final result = await service.login('user@test.com', 'pass');

  expect(result, isTrue);
  expect(await fakeStorage.read(key: 'jwt_token'), equals('abc123'));
});
}
```

10.3 Couverture cible

Couche	Type de tests	Objectif couverture
API Routes	Intégration (Supertest)	80%+
Controllers	Intégration	75%+
Services Flutter	Unitaire (Mockito)	85%+
Middleware Auth	Intégration	100%

11. Guide d'installation

11.1 Prérequis

Avant d'installer CesiZen, vérifiez que les éléments suivants sont présents sur votre machine :

Outil	Version minimale	Vérification
Node.js	18 LTS ou supérieur	<code>node --version</code>
npm	9+ (inclus avec Node)	<code>npm --version</code>
PostgreSQL	14+	<code>psql --version</code>
Flutter SDK	3.10.3+	<code>flutter --version</code>
Git	2.30+	<code>git --version</code>
Android Studio (optionnel)	Electric Eel+	Pour émulateur Android

11.2 Cloner le projet

```
git clone https://github.com/votre-organisation/cesizen_project.git
cd cesizen_project
```

11.3 Configuration de la base de données PostgreSQL

Étape 1 — Créer la base de données

Connectez-vous à PostgreSQL (via psql ou pgAdmin) :

```
CREATE DATABASE cesizen_db;
```

Étape 2 — Initialiser le schéma

Exécutez le script de création des tables :

```
psql -U postgres -d cesizen_db -f database/init_db.sql
```

Étape 3 — Insérer les données de démarrage

Exécutez le script de données initiales (rôles, questions Holmes-Rahe, catégories, activités de démonstration) :

```
psql -U postgres -d cesizen_db -f database/seed.sql
```

Étape 4 — Vérifier l'installation

```
psql -U postgres -d cesizen_db -c "\dt"
```

Vous devriez voir 12 tables : `role`, `utilisateur`, `question`, `emotion`, `diagnostic`, `tracker`, `activity`, `category`, `activity_category`, `favorite`, `article`, `type_dictionnary`.

11.4 Installation et démarrage du backend

Étape 1 — Installer les dépendances

```
cd backend
npm install
```

Étape 2 — Configurer les variables d'environnement

Créez un fichier `.env` à la racine du dossier `backend/` :

```
PORT=3000
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=votre_mot_de_passe_postgres
DB_NAME=cесizen_db
JWT_SECRET=votre_cle_secrete_jwt_longue_et_aleatoire
TOKEN_HEADER_KEY=cесizen_token_header
```

Sécurité : Choisissez une valeur aléatoire et longue pour `JWT_SECRET` (minimum 32 caractères). Ne committez jamais ce fichier dans Git.

Étape 3 — Lancer le serveur

```
# Mode développement (redémarrage automatique)
npm run dev

# Mode production
npm start
```

Sortie attendue :

```
Connecté à la DB
Serveur lancé sur http://localhost:3000
Swagger : http://localhost:3000/api-docs
```

Étape 4 — Vérifier l'API

Ouvrez <http://localhost:3000/api-docs> dans votre navigateur pour accéder à la documentation Swagger interactive.

Étape 5 — Lancer les tests (optionnel)

```
npm test
```

11.5 Installation et démarrage de l'application mobile

Étape 1 — Vérifier l'installation Flutter

```
flutter doctor
```

Résolvez toutes les erreurs signalées avant de continuer.

Étape 2 — Installer les dépendances Flutter

```
cd mobile_app
flutter pub get
```

Étape 3 — Générer les mocks de tests (si besoin)

```
flutter pub run build_runner build --delete-conflicting-outputs
```

Étape 4 — Configurer l'URL de l'API

Ouvrez [mobile_app/lib/config.dart](#) et vérifiez la configuration :

```
// Pour un navigateur web
static const String apiBaseUrlWeb = 'http://localhost:3000/api';

// Pour un émulateur Android
static const String apiBaseUrlAndroid = 'http://10.0.2.2:3000/api';
```

Sur un **appareil physique Android**, remplacez **10.0.2.2** par l'adresse IP locale de votre machine (ex: **192.168.1.100**).

Étape 5 — Lancer l'application utilisateur

```
# Lister les appareils disponibles
flutter devices

# Lancer sur émulateur/appareil
flutter run -t lib/main.dart
```

Étape 6 — Lancer le back-office administrateur

```
flutter run -t lib/main_admin.dart
```

Étape 7 — Lancer les tests Flutter

```
flutter test
```

11.6 Compte administrateur par défaut

Après l'exécution du seed, un compte administrateur doit être créé manuellement via SQL ou via l'API d'inscription (puis mettre à jour le rôle en base) :

```
-- Après inscription via l'API, mettre à jour le rôle
UPDATE utilisateur SET id_role = 1 WHERE email = 'admin@cesizen.fr';
```

11.7 Résolution des problèmes courants

Problème	Solution
ECONNREFUSED 5432	Vérifier que PostgreSQL est démarré : <code>pg_ctl status</code>
invalid password	Vérifier <code>DB_PASSWORD</code> dans le <code>.env</code>
Cannot find module	Relancer <code>npm install</code> dans <code>backend/</code>
flutter doctor erreurs Android	Installer Android SDK via Android Studio
Emulateur ne contacte pas l'API	Utiliser <code>10.0.2.2</code> au lieu de <code>localhost</code>
Token expiré	Se reconnecter : les tokens durent 24h (user) ou 8h (admin)

12. Annexes

12.1 Variables d'environnement complètes

Variable	Description	Valeur par défaut
PORT	Port d'écoute du serveur	3000
DB_HOST	Hôte PostgreSQL	localhost
DB_PORT	Port PostgreSQL	5432
DB_USER	Utilisateur PostgreSQL	postgres
DB_PASSWORD	Mot de passe PostgreSQL	—
DB_NAME	Nom de la base de données	cesizen_db
JWT_SECRET	Clé secrète de signature JWT	—
TOKEN_HEADER_KEY	Clé d'en-tête pour le token	cesizen_token_header

12.2 Dépendances backend

Package	Version	Rôle
express	^5.2.1	Framework HTTP
pg	^8.18.0	Driver PostgreSQL
bcrypt	^6.0.0	Hashage des mots de passe

Package	Version	Rôle
jsonwebtoken	^9.0.3	Génération/validation JWT
cors	^2.8.6	Gestion CORS
dotenv	^17.2.4	Variables d'environnement
swagger-jsdoc	^6.2.8	Documentation API
swagger-ui-express	^5.0.1	Interface Swagger
nodemon	^3.1.11	Redémarrage auto (dev)
jest	^30.3.0	Framework de tests
supertest	^7.2.2	Tests HTTP

12.3 Dépendances Flutter

Package	Version	Rôle
flutter	SDK ^3.10.3	Framework UI
google_fonts	^8.0.1	Polices Google
flutter_secure_storage	^10.0.0	Stockage sécurisé
http	^1.6.0	Client HTTP
youtube_player_flutter	^9.1.3	Lecteur YouTube
mockito	^5.4.4	Mocks pour tests
build_runner	^2.4.8	Génération de code

12.4 Échelle de Holmes-Rahe (extrait)

Le questionnaire de diagnostic est basé sur l'échelle de stress de Holmes-Rahe (1967), un outil cliniquement validé. Voici les 10 événements de vie les plus stressants selon cette échelle :

Rang	Événement	Score
1	Décès du conjoint	100
2	Divorce	73
3	Séparation	65
4	Emprisonnement	63
5	Décès d'un membre proche de la famille	63
6	Blessure ou maladie grave	53
7	Mariage	50
8	Perte d'emploi	47

Rang	Événement	Score
9	Retraite	45
10	Changement de santé d'un proche	44

Le score total est la somme des événements cochés. L'interprétation : score < 100 (faible risque), 100-300 (risque modéré), > 300 (risque élevé de maladie liée au stress).

12.5 Codes de réponse HTTP utilisés

Code	Signification	Utilisation dans CesiZen
200	OK	Succès des opérations GET, PUT, PATCH, DELETE
201	Created	Création réussie (register, create activity, save diagnostic)
400	Bad Request	Champs manquants, email déjà utilisé
401	Unauthorized	Identifiants incorrects, token absent
403	Forbidden	Compte désactivé, rôle insuffisant, token invalide/expiré
404	Not Found	Ressource inexistante
500	Internal Server Error	Erreur base de données ou serveur

Documentation rédigée pour le projet CesiZen — Version 1.0.0 — Mai 2026